> **LAB 10**

Exercise 1: Load Dataset 1.1 Load the dataset

```
from tensorflow.keras.datasets import imdb

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
from tensorflow.keras.datasets import imdb

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
```

```
print(X_train[0])
```

```
[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5, 25, 100, 43, 838, 112, 50, 670, 2, 9,
```

Exercise 2: Understanding Tokenization 2.1 Explain why text must be converted into numbers

Text cannot be directly processed by neural networks, so it is converted into numerical form.

2.2 Display word index dictionary

```
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
fawn : 34701
tsukino : 52006
nunnery : 52007
sonja : 16816
vani : 63951
woods : 1408
spiders : 16115
hanging : 2345
woody : 2289
trawling : 52008
```

2.3 Convert encoded review back to words

```
reverse_word_index = {value: key for key, value in word_index.items()}

decoded_review = ' '.join([reverse_word_index.get(i - 3, '?') for i in X_train[0]])

print(decoded_review)
```

```
? this film was just brilliant casting location scenery story direction everyone's really suited the part they played and yc
```

Exercise 3: Padding Sequences 3.1 Pad all sequences to length = 200

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

max_len = 200

X_train_pad = pad_sequences(X_train, maxlen=max_len)
X_test_pad = pad_sequences(X_test, maxlen=max_len)
```

3.2 Explain why padding is required

RNN models require fixed-length input, so padding ensures all sequences are of equal size.

Exercise 4: Data Visualization 4.1 Print length of first 10 reviews

```
for i in range(10):
    print(len(X_train[i]))
```

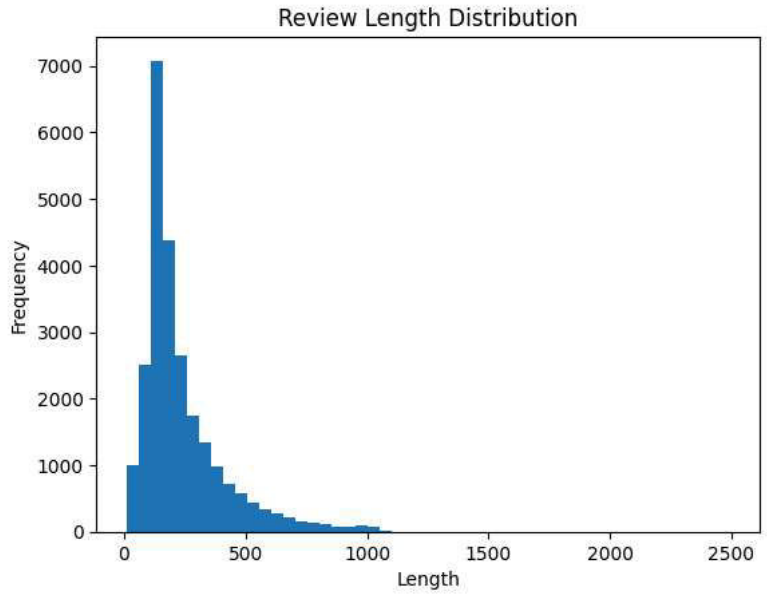
```
218
189
141
550
147
43
123
562
233
130
```

4.2 Plot distribution of review lengths

```
import matplotlib.pyplot as plt

review_lengths = [len(review) for review in X_train]

plt.hist(review_lengths, bins=50)
plt.title("Review Length Distribution")
plt.xlabel("Length")
plt.ylabel("Frequency")
plt.show()
```



4.3 Comment on variability

👉 Review lengths vary significantly, showing uneven text sizes.

Exercise 5: LSTM Model Architecture 5.1 Define the model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model_lstm = Sequential()

model_lstm.add(Embedding(input_dim=10000, output_dim=128, input_length=200))
model_lstm.add(LSTM(64))
model_lstm.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is depr
  warnings.warn(
```

5.2 Compile the model

```
model_lstm.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Exercise 6: Train LSTM Model 6.1 Train the model

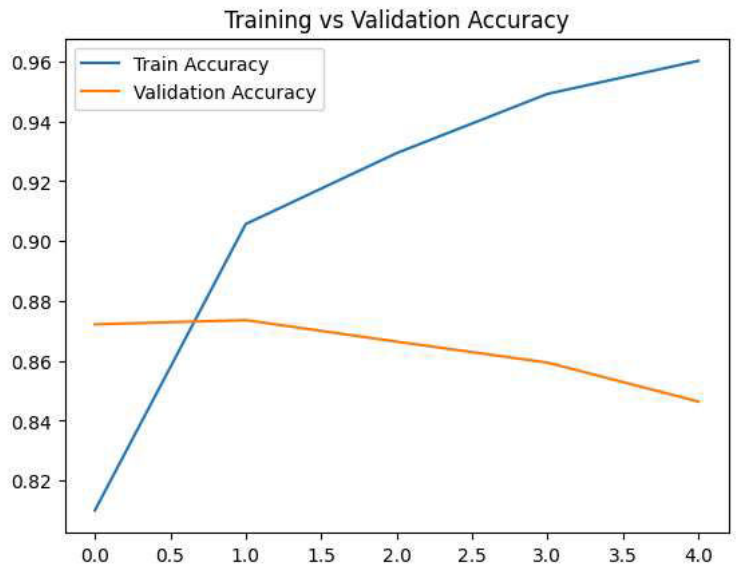
```
history = model_lstm.fit(
    X_train_pad, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 97s 302ms/step - accuracy: 0.8100 - loss: 0.4140 - val_accuracy: 0.8722 - val_loss: 0.3156
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 141s 450ms/step - accuracy: 0.9057 - loss: 0.2446 - val_accuracy: 0.8736 - val_loss: 0.3121
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 88s 280ms/step - accuracy: 0.9294 - loss: 0.1917 - val_accuracy: 0.8664 - val_loss: 0.3549
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 88s 280ms/step - accuracy: 0.9492 - loss: 0.1421 - val_accuracy: 0.8594 - val_loss: 0.3987
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 85s 272ms/step - accuracy: 0.9603 - loss: 0.1106 - val_accuracy: 0.8464 - val_loss: 0.4691
```

6.2 Plot training vs validation accuracy

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.legend()
plt.title("Training vs Validation Accuracy")
plt.show()
```



Exercise 7: Model Evaluation 7.1 Evaluate on test dataset

```
loss, accuracy = model_lstm.evaluate(X_test_pad, y_test)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 30s 38ms/step - accuracy: 0.8432 - loss: 0.4736
```

7.2 Report test accuracy

```
print("Test Accuracy:", accuracy)
```

```
Test Accuracy: 0.8432000279426575
```

7.3 Compare training vs validation

👉 Training accuracy is usually higher; validation shows generalization.

Exercise 8: Confusion Matrix 8.1 Predict sentiment

```
y_pred = model_lstm.predict(X_test_pad)
y_pred = (y_pred > 0.5).astype(int)
```

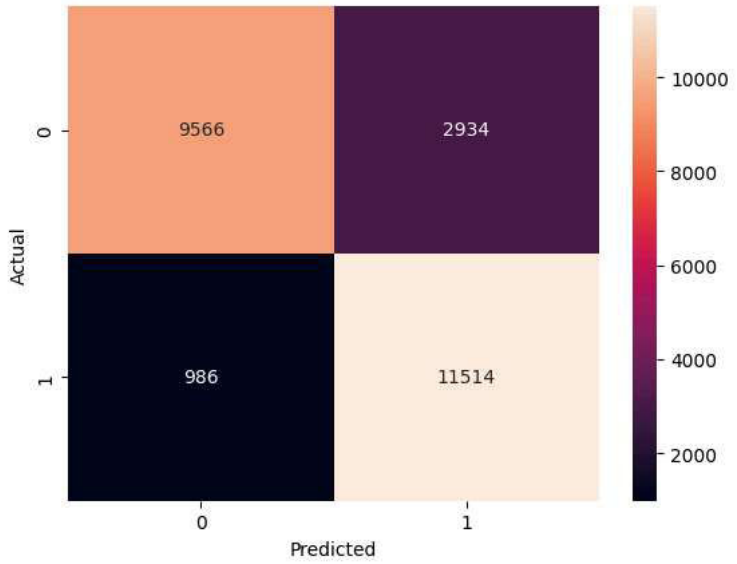
```
782/782 ━━━━━━━━━━━━━━━━━━━━ 29s 37ms/step
```

8.2 Create confusion matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



8.3 Interpretation

👉 False positives = predicted positive but actually negative 👉 False negatives = predicted negative but actually positive

Exercise 9: Bidirectional LSTM 9.1 Define model

```
from tensorflow.keras.layers import Bidirectional

model_bilstm = Sequential()

model_bilstm.add(Embedding(10000, 128, input_length=200))
model_bilstm.add(Bidirectional(LSTM(64)))
model_bilstm.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is depr
  warnings.warn(
```

9.2 Compile model

```
model_bilstm.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

9.3 Train model

```
history_bi = model_bilstm.fit(
    X_train_pad, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 171s 516ms/step - accuracy: 0.7596 - loss: 0.5002 - val_accuracy: 0.7680 - val_loss: 0.5053
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 161s 515ms/step - accuracy: 0.8612 - loss: 0.3332 - val_accuracy: 0.8472 - val_loss: 0.3529
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 160s 512ms/step - accuracy: 0.9133 - loss: 0.2284 - val_accuracy: 0.8348 - val_loss: 0.3587
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 167s 533ms/step - accuracy: 0.9323 - loss: 0.1807 - val_accuracy: 0.8580 - val_loss: 0.3466
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 196s 513ms/step - accuracy: 0.9535 - loss: 0.1303 - val_accuracy: 0.8532 - val_loss: 0.3749
```

Exercise 10: Model Comparison 10.1 Evaluate both models

```
loss1, acc1 = model_lstm.evaluate(X_test_pad, y_test)
loss2, acc2 = model_bilstm.evaluate(X_test_pad, y_test)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 29s 38ms/step - accuracy: 0.8432 - loss: 0.4736
782/782 ━━━━━━━━━━━━━━━━━━━━ 51s 65ms/step - accuracy: 0.8530 - loss: 0.3758
```

10.2 Display comparison

```
print("LSTM Accuracy:", acc1)
print("Bidirectional LSTM Accuracy:", acc2)
```

```
LSTM Accuracy: 0.8432000279426575
Bidirectional LSTM Accuracy: 0.8529599905014038
```

Exercise 11: Hyperparameter Experiment 11.1 Modify parameters

```
from tensorflow.keras.layers import Dropout

model_exp = Sequential()

model_exp.add(Embedding(10000, 256, input_length=300))
model_exp.add(LSTM(64))
model_exp.add(Dropout(0.5))
model_exp.add(Dense(1, activation='sigmoid'))

model_exp.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model_exp.fit(X_train_pad, y_train, epochs=7, batch_size=64, validation_split=0.2)
```

```
Epoch 1/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 121s 378ms/step - accuracy: 0.7840 - loss: 0.4479 - val_accuracy: 0.8570 - val_loss: 0.3527
Epoch 2/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 116s 372ms/step - accuracy: 0.9003 - loss: 0.2552 - val_accuracy: 0.8696 - val_loss: 0.3440
Epoch 3/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 144s 378ms/step - accuracy: 0.9355 - loss: 0.1811 - val_accuracy: 0.8504 - val_loss: 0.4149
Epoch 4/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 117s 375ms/step - accuracy: 0.9507 - loss: 0.1385 - val_accuracy: 0.8714 - val_loss: 0.4600
Epoch 5/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 122s 389ms/step - accuracy: 0.9633 - loss: 0.1025 - val_accuracy: 0.8572 - val_loss: 0.4209
Epoch 6/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 117s 374ms/step - accuracy: 0.9724 - loss: 0.0812 - val_accuracy: 0.8666 - val_loss: 0.5336
Epoch 7/7
313/313 ━━━━━━━━━━━━━━━━━━━━ 121s 388ms/step - accuracy: 0.9779 - loss: 0.0646 - val_accuracy: 0.8634 - val_loss: 0.5093
<keras.src.callbacks.history.History at 0x7d2d7e258ad0>
```

Exercise 12: Analysis 12.1 Embedding layer

ANS: Converts words into dense vectors

12.2 Padding

ANS: Ensures equal input length

12.3 Bidirectional LSTM

ANS: Captures past + future context

12.4 Small sequence length

ANS: Important information may be lost

12.5 Sigmoid activation

ANS: Outputs probability (0–1)